

Lecture 20:
Deep Learning
Advancement in Convolutional Neural
Networks

ML/AI for Engineering Applications
ENV 6932

Sara Behdad
Fall 2025

Outline

- **Recall:** Activation Functions, Loss Functions, Gradient Descent, Back Propagation, Bias-Variance in NNs, Transfer Learning
- Innovations in Convolutional Neural Networks

Advanced Deep Learning Architectures

- **Early CNNs (AlexNet, VGG):**
 - Hierarchical feature extraction via stacked convolution layers.
 - Great success on ImageNet but limited scalability.
- **Challenges:**
 - Vanishing gradients when depth increases.
 - Overfitting due to large parameters.
 - Fixed receptive field will result to limited context capture.

The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

The Four Innovations

1	Innovation	Representative Architecture	Core Concept / Breakthrough
	Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
	Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
	Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
	Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

When Deeper Networks Fail

- **Why Deeper CNNs Do not Always Perform Better?**
- Early CNNs (AlexNet, VGG) achieved strong results by increasing depth.
- But beyond ~20 layers, training accuracy starts to degrade.
- **The issue:** vanishing gradients: gradients get smaller as they backpropagate. The deeper the model, the harder to optimize.

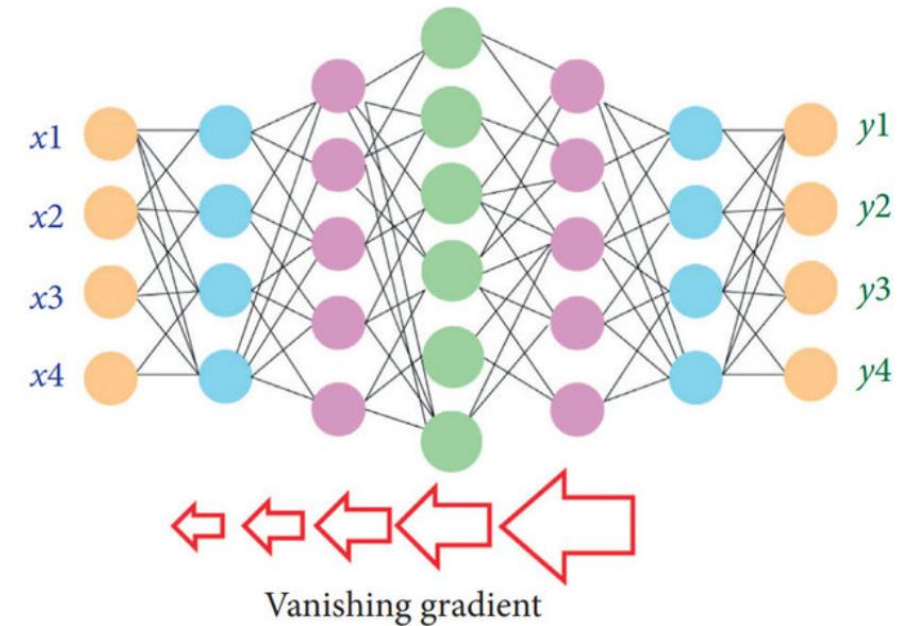
$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_L} \prod_{i=1}^L \frac{\partial x_i}{\partial x_{i-1}}$$

When Deeper Networks Fail

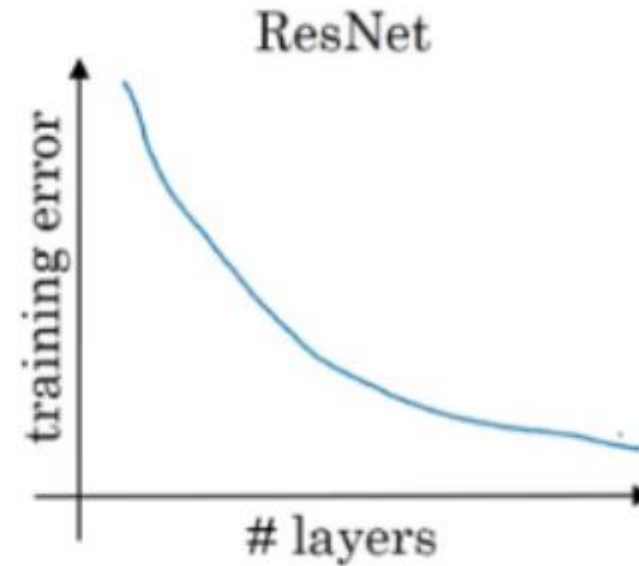
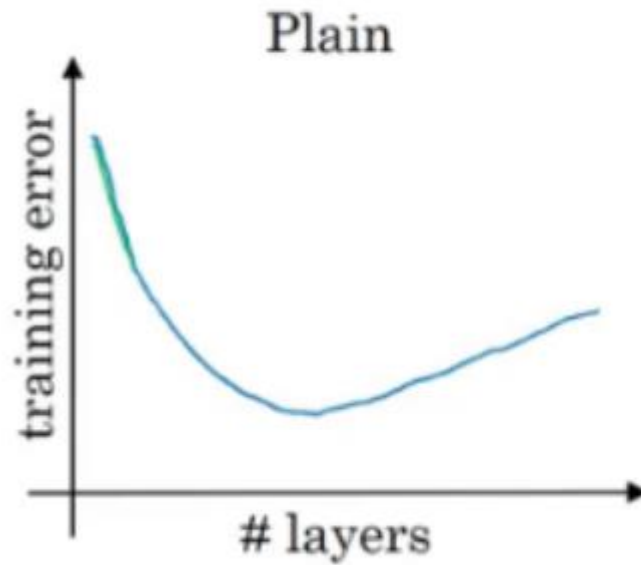
- Why Deeper CNNs Do not Always Perform Better?

$$\frac{\partial L}{\partial x_0} = \frac{\partial L}{\partial x_L} \prod_{i=1}^L \frac{\partial x_i}{\partial x_{i-1}}$$

- If each $\frac{\partial x_i}{\partial x_{i-1}} < 1$, the product will be 0.



Plain CNN vs ResNet



The Degradation Problem

- Adding more layers should improve representational power, but does not.
- Training error increases with depth beyond a threshold.
- Indicates optimization difficulty, not overfitting.

How can we design networks that get *deeper* without hurting training?

What Are We Trying to Learn in a Deep Network?

- Every neural network is trying to approximate some function $H(x)$:
$$y = H(x)$$

where x is the input feature map, and y is the desired output (after a few layers).

- In a traditional CNN, each new layer tries to completely transform the previous features into something new:

$$H(x) = Layer_n(Layer_{n-1}(\dots(x)\dots))$$

That means every new layer starts from scratch. It must ‘figure out’ what to do with the input on its own, even if the input is already *pretty good*.

The Problem: Sometimes the Best Mapping Is ‘Do Nothing’

In very deep networks, not every layer needs to change the input drastically. Sometimes the optimal behavior is:

‘Just pass the information forward without change.’

For example:

- Early layers extract edges and colors.
- Middle layers might not need to alter them, they might just forward these basic features up.

So ideally, those layers should learn the identity mapping:

$$H(x) = x$$

Meaning: output = input, no change.

But Traditional CNNs Struggle to Learn Identity

- In a standard CNN, each layer uses convolution + ReLU:

$$H(x) = \sigma(Wx + b)$$

- If we want $H(x) = x$, then we would need:
 - W to behave like the identity matrix.
 - $b = 0$.
 - ReLU not to zero out negative values.
- That is **very hard** to learn through gradient descent because:
 - There is no explicit shortcut from input to output.
 - The optimizer must adjust many small weights just to simulate passing input unchanged.

The ResNet Solution: Separate the Identity from the Learning

- ResNet introduces a *skip connection*:

$$H(x) = F(x) + x$$

- Here:
 - x : the *input*, sent directly to the output (the identity path).
 - $F(x)$: a *residual function*, a small correction.
- So the network no longer needs to learn the entire $H(x)$.
It only needs to learn how to change the input slightly.

Residual Learning: Learn the Change, Not the Whole Function

- Instead of learning $H(x)$ directly, ResNet learns how the output differs from the input.

$$H(x) = F(x) + x$$

- x : input (identity path)
 - $F(x)$: residual (correction or change)
- If $F(x) = 0$ block acts as identity.
- If $F(x) \neq 0$ block learns meaningful adjustment.

Not an Assumption: A Choice

- **Clarification**

- The network is free to decide when to keep or modify information.
- The skip (identity) path simply gives it that option.
 - If modification is helpful, $F(x)$ grows large.
 - If features are already good, $F(x)$ stays small, output $\approx$ input.

- **In Training**

- At initialization, weights are small, $F(x) \approx 0$, smooth start.
- Gradually, only useful residuals are learned.

What Happens in Practice

In Practice:

- Early layers: $F(x)$ small, identity dominates (keep edges/textures).
 - Middle layers: moderate residuals, combine and reshape features.
 - Later layers: large residuals, prepare for classification.
-
- ResNet does not force identity, it enables it.

$$x_{l+1} = x_l + F(x_l, W_l)$$

What is ResNet?

- ResNet (**Residual Network**) is a deep CNN that uses skip connections to solve vanishing gradient and degradation problems.
- Introduced by He et al., 2015, assisting networks with hundreds of layers to train more effectively.
- Each layer (or block) learns **only residual changes to the input**, not a brand-new mapping

Why Skip Connections Matter

- Skip connections provide a direct path for both features and gradients.
- Prevent information loss in very deep networks.
- Allow the model to learn small corrections instead of full transformations.

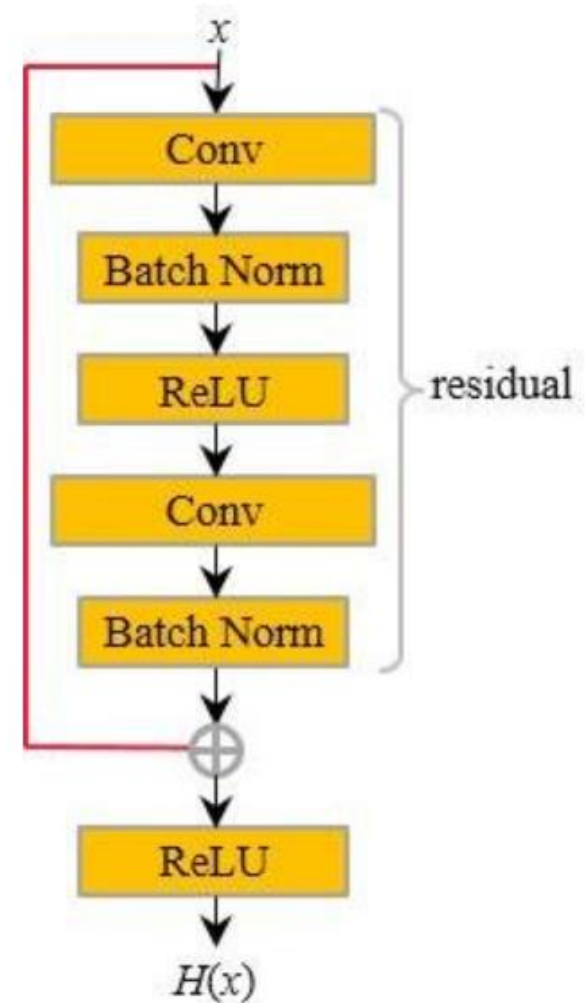
$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \left(1 + \frac{\partial F(x)}{\partial x} \right)$$

The Building Unit of ResNet

- Structure:
 - Two 3×3 convolutional layers + BatchNorm + ReLU.
 - Skip connection adds input to the output of these layers.

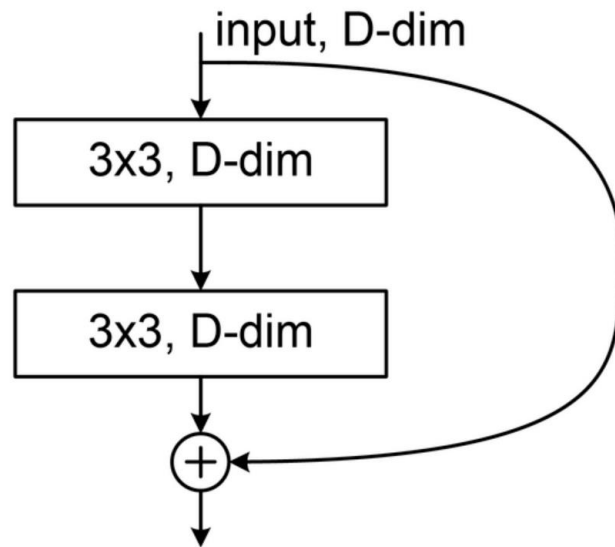
Used in **ResNet-18** and **ResNet-34**.

Single Residual Block

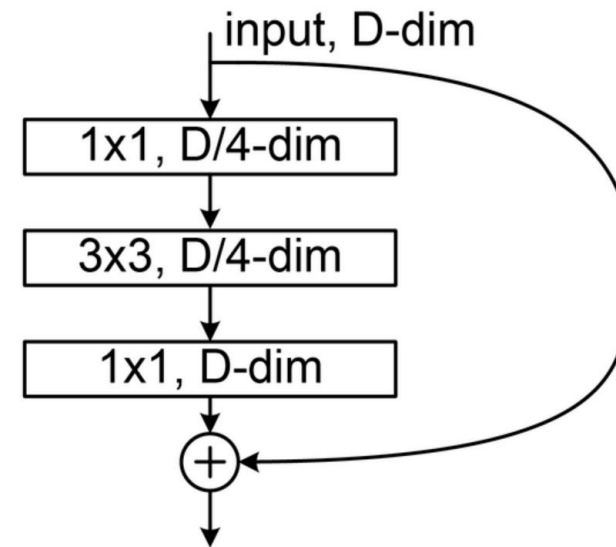


The Bottleneck Block (for Deeper ResNets)

- Deeper networks (ResNet-50, 101, 152) are computationally expensive.
- **Solution:** Add 1×1 (kernel size) convolutions to reduce and then expand channel depth around the 3×3 layer.



Single Residual Block



Bottleneck Block

$$y = W_3 \sigma(BN(W_2 \sigma(BN(W_1 x)))) + x$$

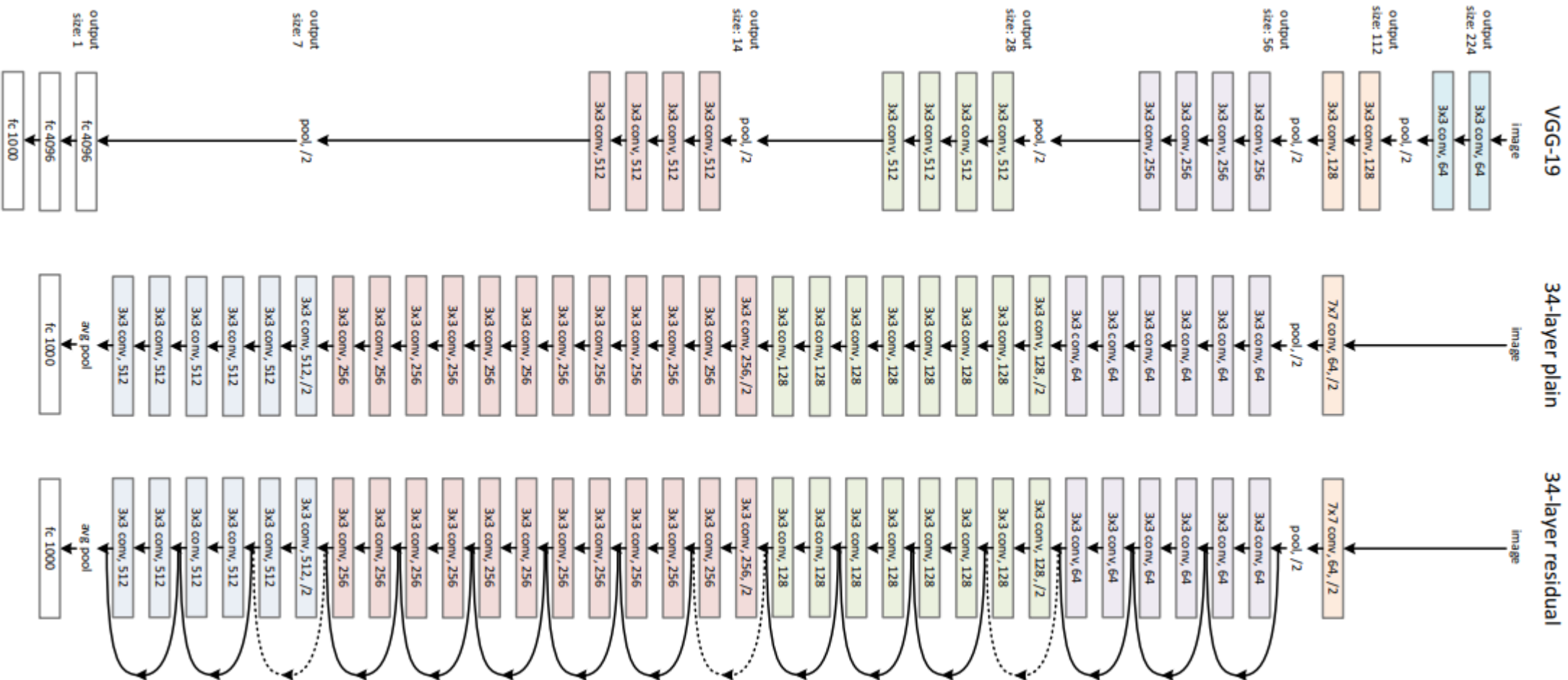
Why the 1×1 Convolution Helps

Operation	Params Formula	Example Params (Cin=Cout=256)
3×3 Conv	$(9 \times 256 \times 256)$	589,824
1×1 Conv	$(1 \times 256 \times 256)$	65,536

In Bottleneck:

- 1×1 reduces channels (e.g., 256 to 64).
- 3×3 works on fewer channels.
- Final 1×1 expands back (64 to 256).

ResNet-34



The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

2

Why Go Beyond Skip Connections?

- ResNet allowed gradients and information to flow better through *addition-based* shortcuts.
- But still:
 - Each block only connects to its immediate previous block.
 - Earlier features can still fade as depth increases.
 - Information is not fully reused, later layers should relearn similar features.

Can we design a network where *every layer directly accesses all previous features*?

The Core Idea of Dense Connectivity

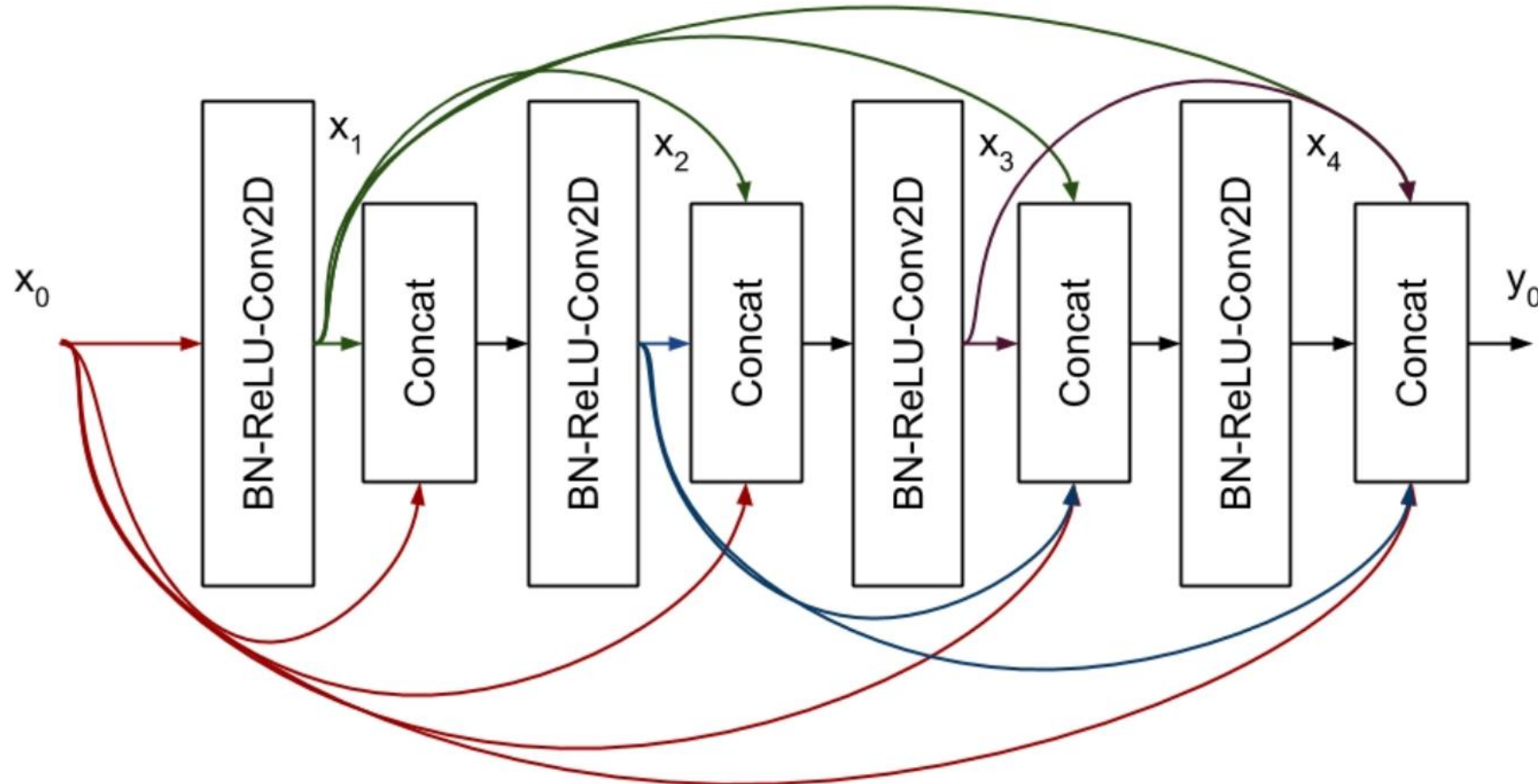
- Connect Every Layer to Every Other Layer
- DenseNet introduces **dense connections**, where each layer receives as input the *concatenation* of all preceding layers' feature maps.

$$x_l = H_l([x_0, x_1, x_2, \dots, x_{l-1}])$$

- $[]$ = concatenation along the channel dimension.
 - H_l = a composite function (BN --- ReLU --- Conv).
 - Every layer **reuses** earlier features directly, no relearning.
- **Key Difference from ResNet:**
 - ResNet: adds features: $x + F(x)$
 - DenseNet: concatenates features: $[x, F(x)]$

Inside a Dense Block

A **Dense Block** is a group of layers with dense connectivity.



A 4-layer Dense block in DenseNet.

Growth Rate: Controlling the Expansion

- Concatenation adds channels each time, output size keeps growing.

- **Solution:**

Use a growth rate k : number of new feature maps each layer adds.

If block has L layers and input = C_{in} channels:

$$C_{out} = C_{in} + L \times k$$

- Typical Values: $k = 12, 24, 32$
- Each layer adds only k new features.
- Earlier features remain accessible without re-learning.

Transition Layers

Keeping DenseNet Manageable

Between dense blocks, we insert transition layers to reduce spatial and channel dimensions.

Structure:

BN, 1×1 Conv (compression, 2×2 Average Pool

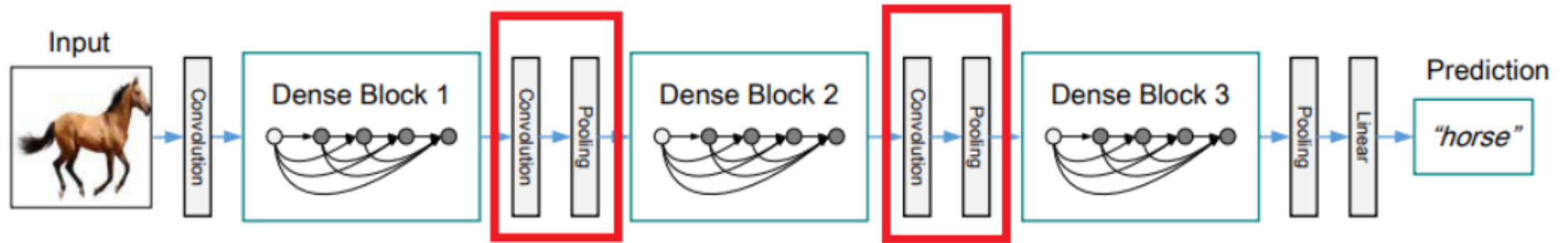
Roles:

- 1×1 Conv: compress channels (reduce width).
- Avg Pooling: halve spatial resolution.
- Controls model size and memory footprint.

Compression Factor (θ):

$$C_{out} = \theta C_{in}, \quad \text{typically } \theta = 0.5.$$

Transition Layers



A deep DenseNet with three dense blocks and two transition layers

DenseNet vs ResNet: Different Connectivity Philosophies

Feature	ResNet	DenseNet
Skip Type	Addition ($(x + F(x))$)	Concatenation ($([x, F(x)])$)
Info Flow	Nearest neighbor	All previous layers
Gradient Flow	Through shortcuts	Through every feature map
Feature Reuse	Limited	Strong (explicit)
Parameter Efficiency	Moderate	Often fewer parameters
Example	ResNet-50	DenseNet-121

The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
3 Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

Parallel Filters and the Inception Architecture

- One Filter Size Cannot Fit All
- Why We Need Multiple Scales of Vision?
 - Traditional CNN layers use a *fixed* filter size (e.g., 3×3).
 - But visual patterns vary:
 - Small features: edges, textures
 - Large features: objects, contexts
- A single kernel size cannot capture both *local* and *global* context properly.

Can a network see at multiple scales simultaneously?

The Inception Idea

- Multi-Scale Processing Through Parallel Filters
- Inception modules process the same input through **multiple convolutional paths in parallel**, each with different kernel sizes.

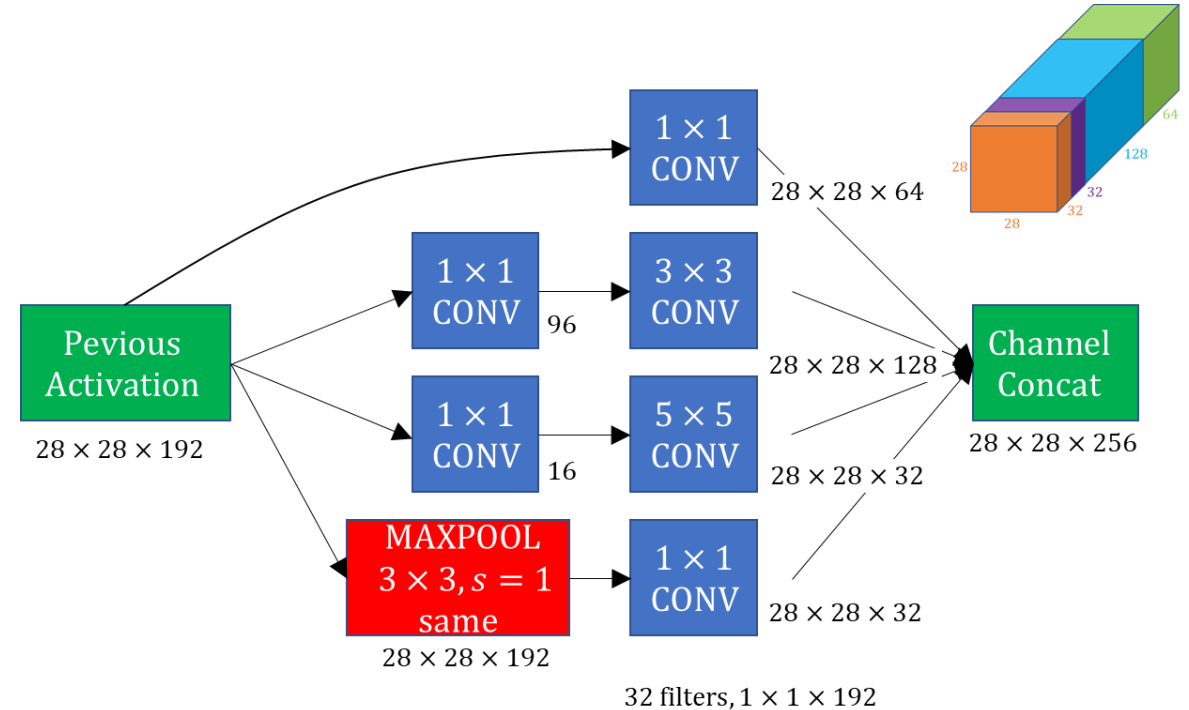
$$\text{Output} = [\text{Conv}_{1 \times 1}(x), \text{Conv}_{3 \times 3}(x), \text{Conv}_{5 \times 5}(x), \text{Pooling}(x)]$$

Concatenate all outputs along the channel dimension.

- Each path captures information at a different receptive field size.
- The network learns *which scale matters* for each feature.

Architecture of an Inception Module

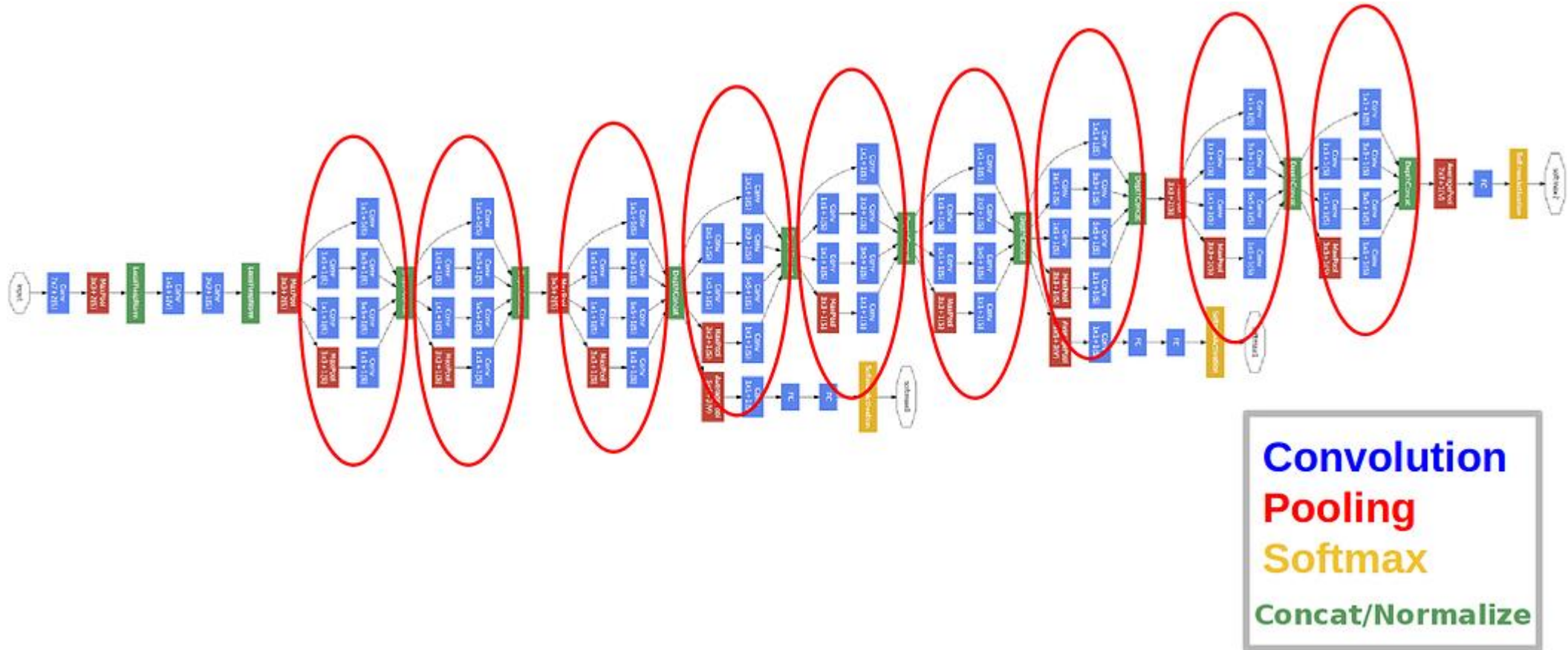
- Branch 1:
1×1 Convolution
- Branch 2:
1×1 Convolution → 3×3 Convolution
- Branch 3:
1×1 Convolution → 5×5 Convolution
- Branch 4:
3×3 Max Pooling → 1×1 Convolution



The outputs from **all four branches** are **concatenated depth-wise** (along the channel axis):

$$\text{Output} = [\text{Branch}_1, \text{Branch}_2, \text{Branch}_3, \text{Branch}_4]$$

GoogleNet



The Four Innovations

Innovation	Representative Architecture	Core Concept / Breakthrough
Skip Connections	ResNet (2015)	Identity mapping for stable deep learning
Dense Connectivity	DenseNet (2017)	Feature reuse through concatenation
Parallel Filters	Inception / GoogLeNet (2015)	Multi-scale spatial feature extraction
4 Self-Attention	Vision Transformer (ViT, 2021)	Global dependency modeling without convolutions

Self-Attention and Vision Transformers (ViT)

Why Move Beyond Convolutions?

- **Problems with CNNs:**

- Convolutions have a local receptive field, they only capture nearby spatial relations.
- Deeper layers expand context, but long-range dependencies is hard to learn.
- Filter weights are fixed once trained: not adjustable to input content.
- CNNs focus on spatial hierarchies, not global relationships.

Can we design a model that directly attends to relationships between any two image regions, no matter how far apart?

Tokens Are the Input Units for Attention

- Split the image into small patches
- Example: 224×224 image: divide into 16×16 pixel patches, you get $(224/16)^2 = 14 \times 14 = 196$ patches.
- Each 16×16 patch becomes one token.
- The Transformer does not see 'the image' or 'the sentence' as a whole. it only sees a sequence of tokens, similar to:

$$[x_1, x_2, x_3, \dots, x_n]$$

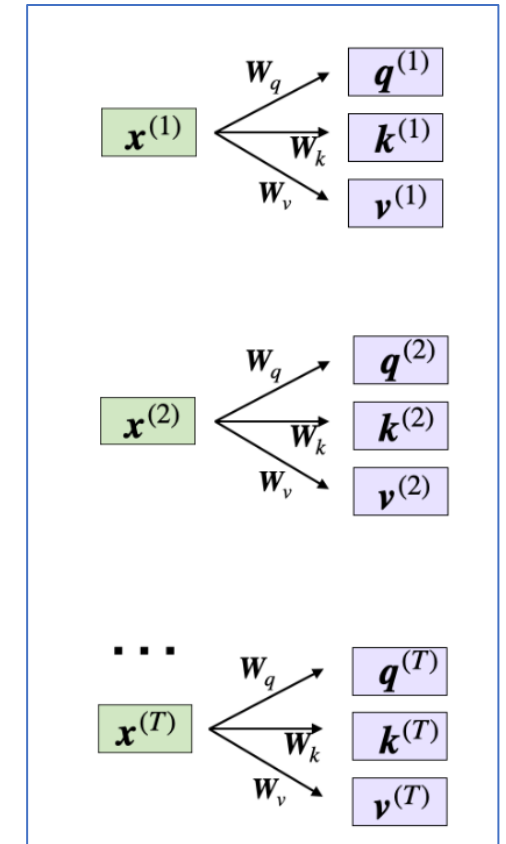
- Each token will:
 - Create its own **query (Q)**: what I am looking for
 - Create its own **key (K)**: what I offer
 - Create its own **value (V)**: what information I contain

Learning Relationships, Not Just Filters

- **Concept: Self-Attention**
- Each pixel (or patch) can attend to every other pixel in the image.
The model learns *which regions are important for interpreting each patch*.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- where
 - Q = query vectors,
 - K = key vectors,
 - V = value vectors,
 - d_k = dimension scaling factor.



How ViT Processes an Image

- Split the image into patches (e.g., 16×16 pixels each).
- Flatten each patch into a 1D vector.
- Project each vector linearly into an embedding space (similar to word embeddings in NLP).
- Add positional encoding (so the model knows patch order and spatial layout).
- Feed into a Transformer Encoder (a stack of multi-head attention + feedforward layers).
- Aggregate output (via classification token), final class prediction.

What are Q, K, and V?

- Every input token (in NLP, a word; in ViT, an image patch) is represented as a vector, an embedding.
- From each embedding, the network computes three different projections:

Symbol	Name	Role
Q	Query	Represents what this token is looking for
K	Key	Represents what other tokens offer
V	Value	Represents the actual information to transfer

- These come from learnable linear transformations:

What are Q, K, and V?

Symbol	Name	Role
Q	Query	Represents what this token is looking for
K	Key	Represents what other tokens offer
V	Value	Represents the actual information to transfer

- These come from learnable linear transformations:

$$Q = XW^Q, \quad K = XW^K, \quad V = XW^V$$

- where X is the input (matrix of token embeddings) and W^Q, W^K, W^V are learned weight matrices.
- So each token produces one query vector, one key vector, and one value vector.

Computing Relationships QK^T

- Now, we measure *how relevant each token is to every other token*.
- QK^T is a similarity matrix:
 - Each element (i, j) = similarity between the query of token i and the key of token j.
 - If two vectors point in a similar direction (high dot product), it means token i should pay more attention to token j .
- This gives us an $n \times n$ matrix if there are n tokens: it tells us **who attends to whom**.

The Scaling Factor $\sqrt{d_k}$

- Each query and key vector has a dimension d_k .
When we take a dot product between two high-dimensional vectors, the result can get large just because of dimensionality, not because of strong correlation.
- To keep the numbers stable and prevent extremely large values that would dominate after the softmax, we normalize by dividing by $\sqrt{d_k}$.

$$\frac{QK^T}{\sqrt{d_k}}$$

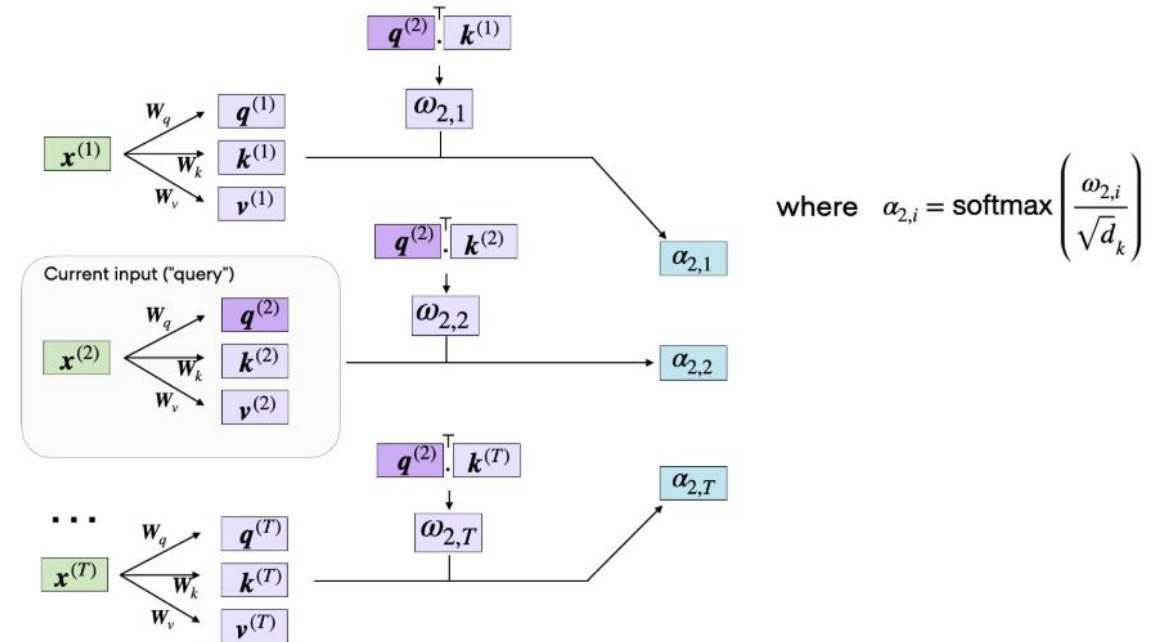
- is just a scaled version of the similarity matrix. It keeps training numerically stable.

Softmax: Turning Scores into Weights

- The softmax function converts each row (all similarities for a given query) into probabilities that sum to 1.
- So for token i , the softmax gives you a set of weights:

$$\text{Attention Weights}_i = \text{softmax}\left(\frac{Q_i K^T}{\sqrt{d_k}}\right)$$

- High weight: token i focuses strongly on token j
- Low weight: token i mostly ignores token j



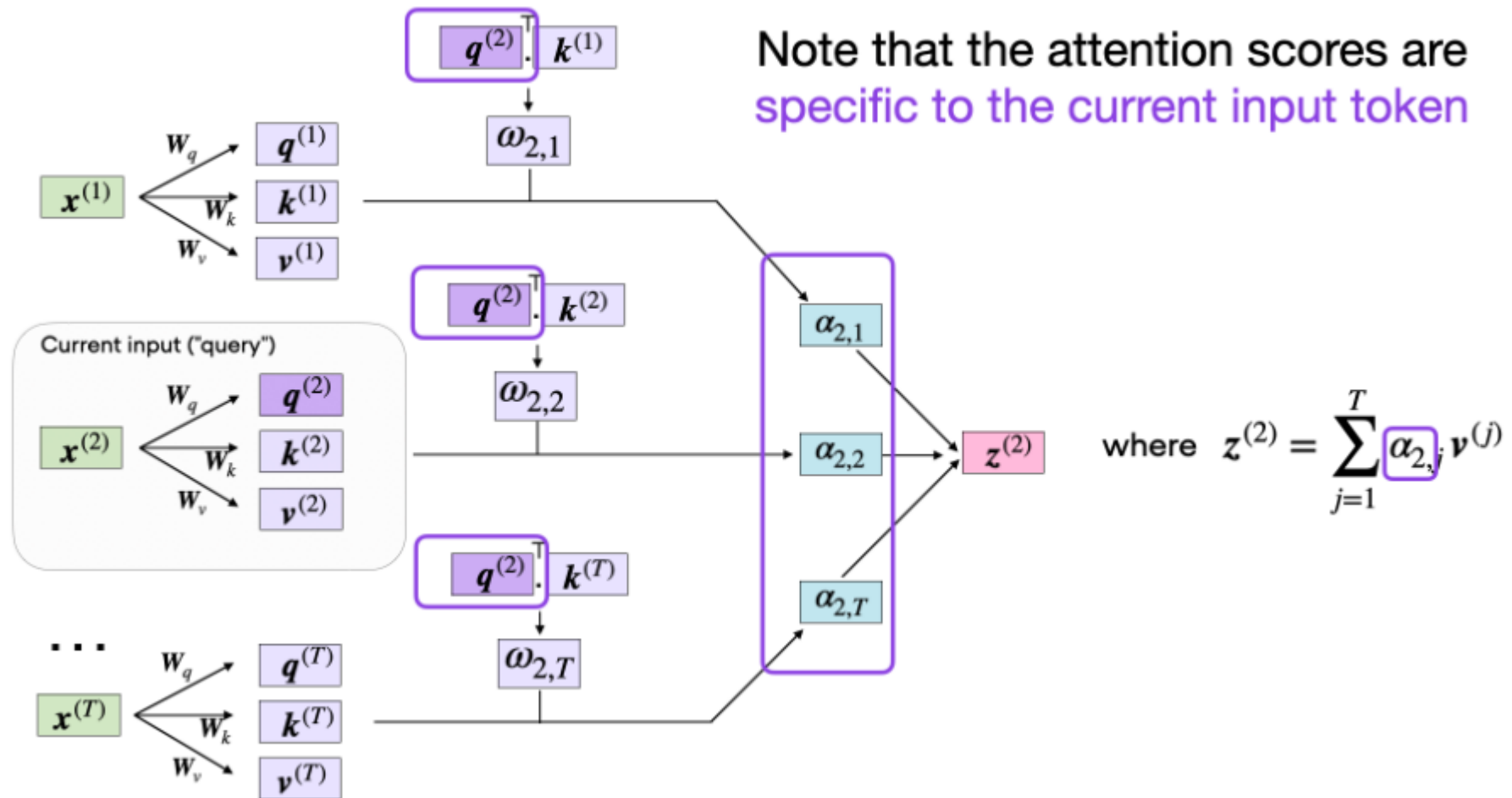
Weighted Sum with V

- Finally, we multiply those attention weights by the values (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Each token's new representation = weighted average of all the value vectors,
where the weights tell us how important each other token is to it.

Attention-weighted version of original query for the second input element



Intuitive Example

Imagine we are processing an image patch representing a cat's **eye**:

- The '**query**' from the eye patch asks: *Which other patches help me understand what I am?*
- The model computes dot products with '**key**' vectors of all patches (nose, fur, background...).
- High similarity with nearby 'face' patches, high attention weights there.
- The model gathers corresponding **value** information from those patches, combines them, output embedding now represents 'eye with context of face.'

Putting It All Together

Step	Operation	Meaning
1	$Q, K, V = XW^Q, XW^K, XW^V$	Linear projections of input
2	QK^T	Pairwise similarity between tokens
3	Divide by $\sqrt{d_k}$	Normalize scale of similarities
4	Softmax	Convert similarities $\rightarrow$ probabilities (attention weights)
5	Multiply by V	Aggregate information from all tokens based on attention

In Short:

QK^T : “How much should I attend to you?”

Softmax(·): “Turn similarities into probabilities.”

$\times V$: “Use those weights to blend your information into mine.”

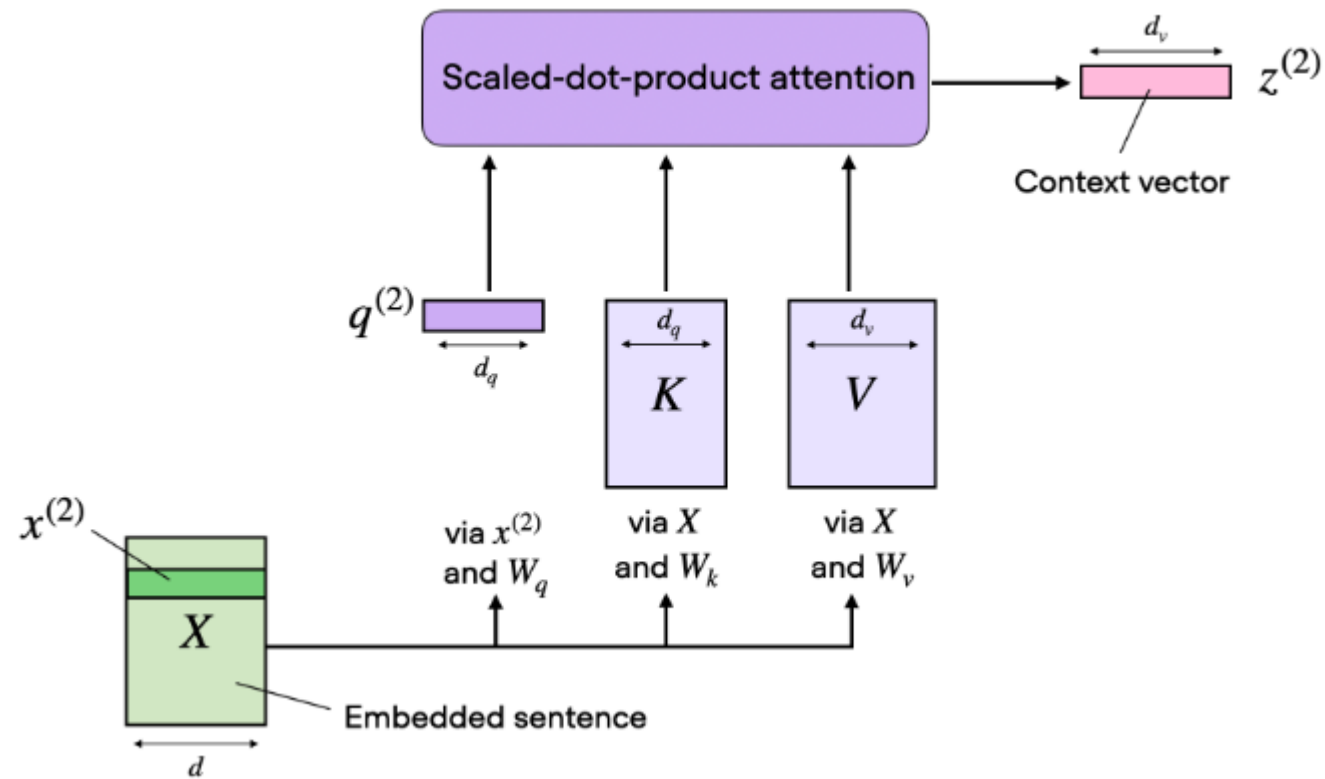
Analogy: Students in a Classroom

- Think of an image (or sentence) as a classroom, and each token is a student.
- Each student (token) has their own notes (V).
- Each student also asks questions (Q) and offers answers (K).
- During discussion (self-attention):
 - Each student looks at everyone else's answers (QK^T).
 - Decides whose answers are relevant (softmax).
 - Mixes relevant notes into their own ($\times V$).
- Result: Every student's understanding now includes the entire class's perspective.

Several Concepts

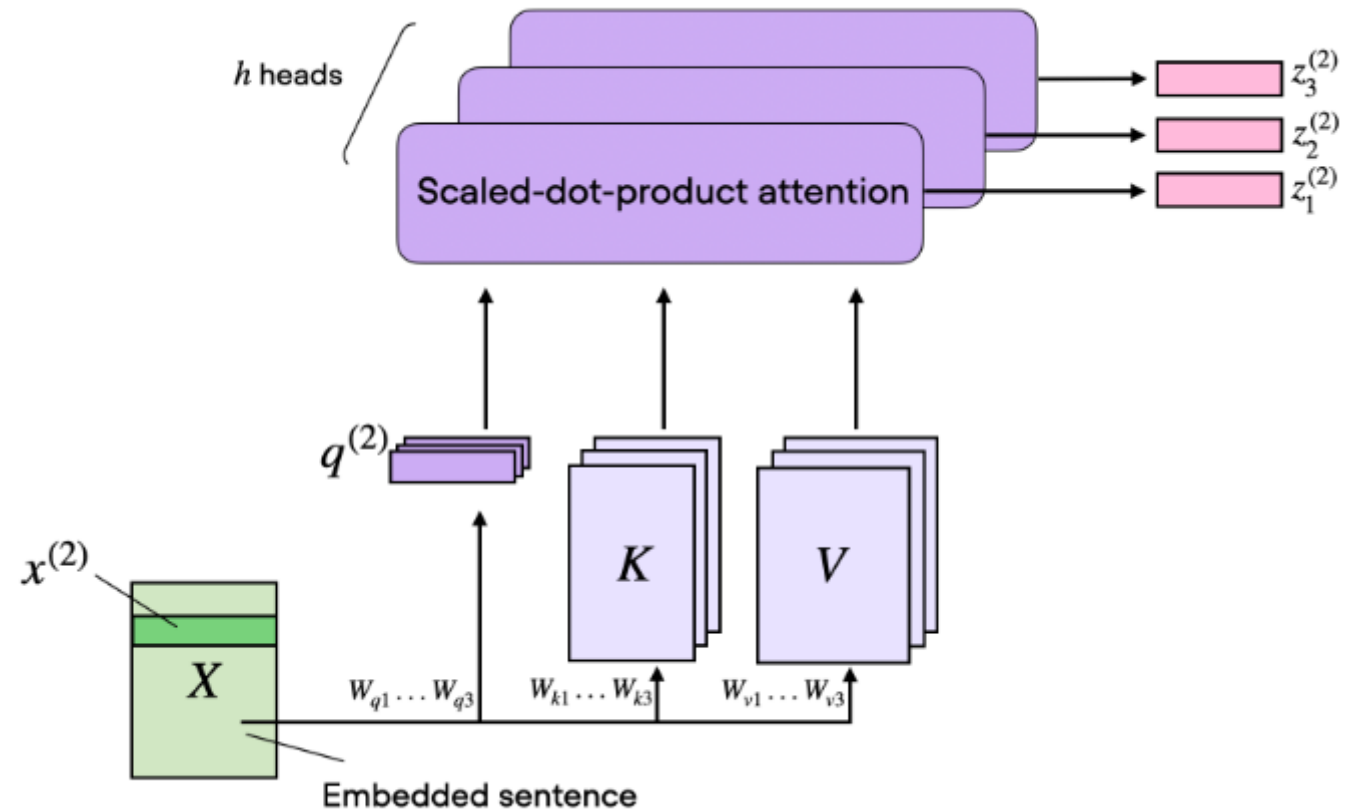
- Multi-head attention
- Cross-attention

Self-attention Mechanism



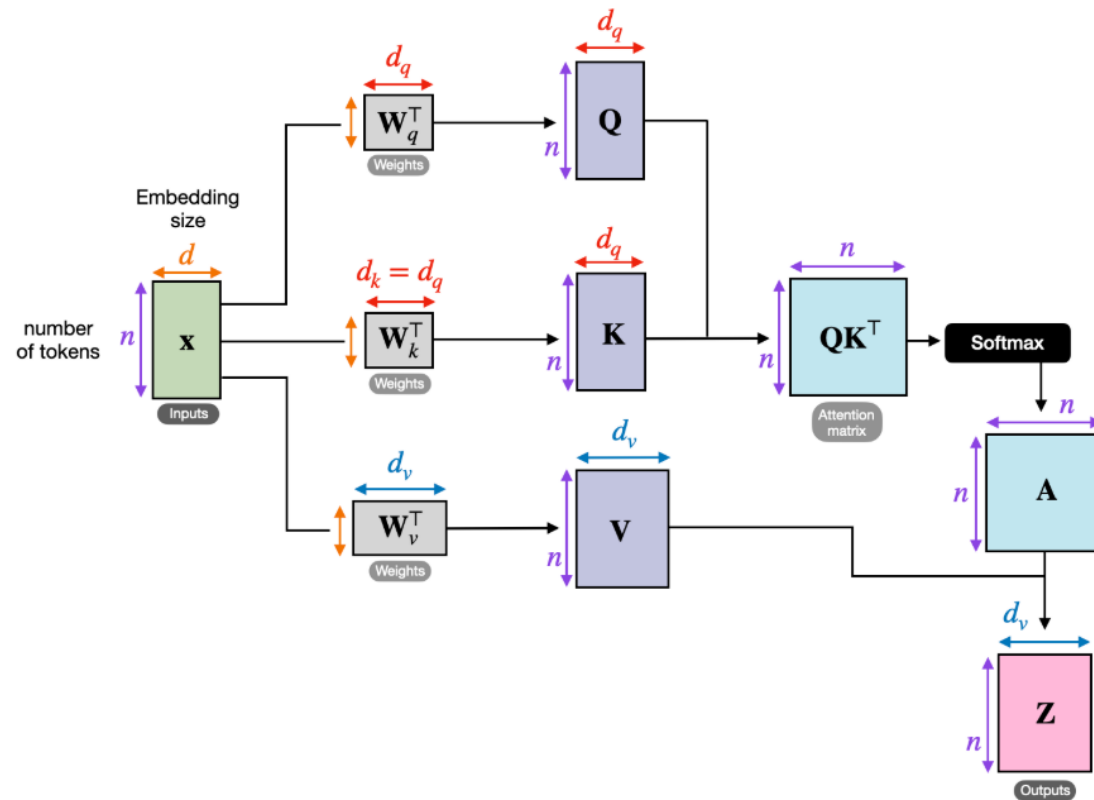
Multi-head Attention

This concept is similar to the use of multiple kernels in convolutional neural networks.



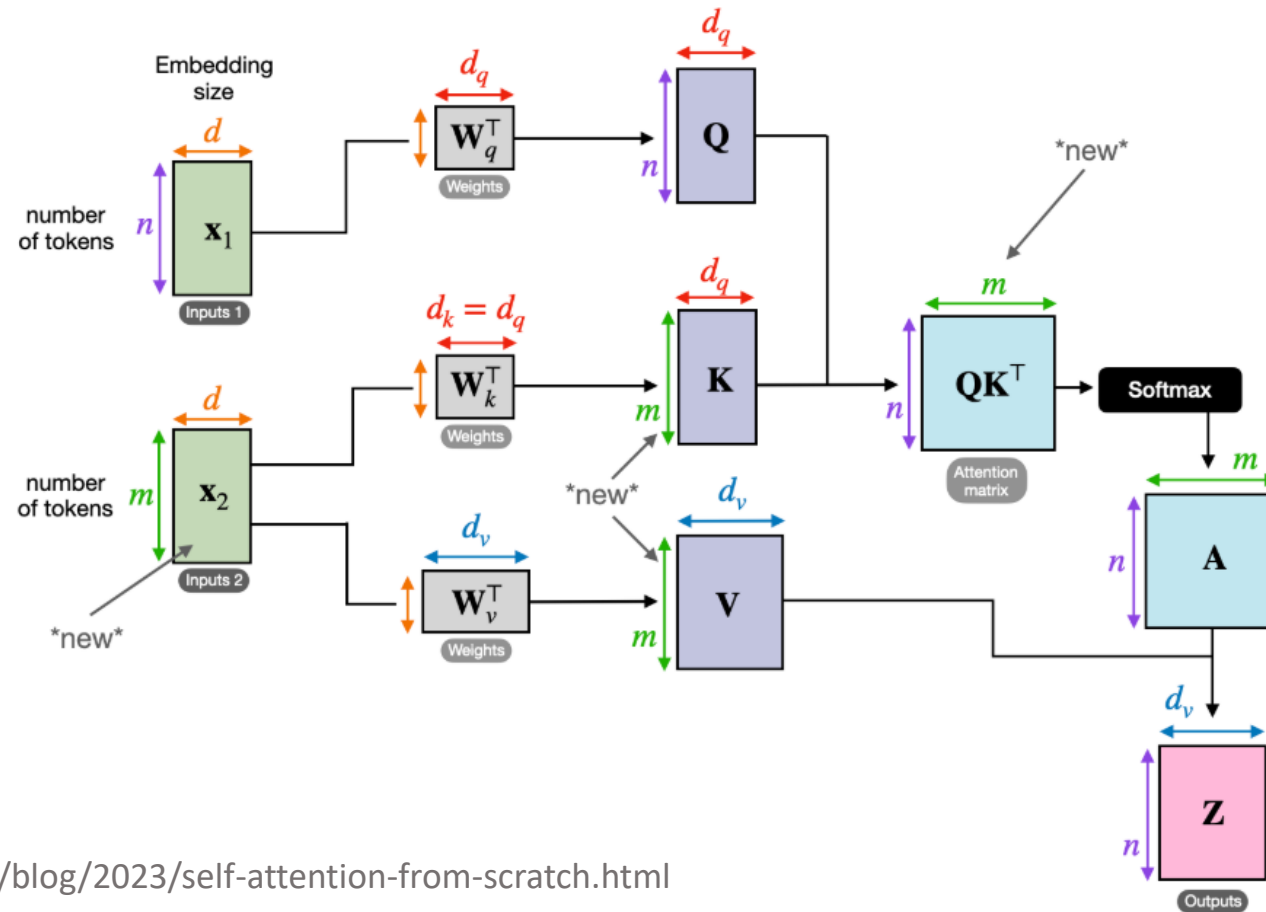
Self-attention

- In self-attention, we work with the same input sequence.



Cross-attention

- In cross-attention, we mix or combine two *different* input sequences



From Classification to Segmentation

- **Why Move Beyond Classification Networks?**
- Models like ResNet, DenseNet, and ViT output a single label per image.
- But many real-world problems require dense, spatial predictions:
 - Semantic segmentation: classify every pixel
 - Object detection: localize multiple objects
 - Depth estimation: continuous map
 - Medical imaging: outline organs, cells

We need architectures that can predict for every pixel, not just the whole image.

Revisiting CNN Output Structure

- Why Standard CNNs Cannot Do Segmentation Directly
- **Standard CNN pipeline:**
 - Pooling and FC layers destroy spatial resolution.
 - The network no longer knows *where* features came from.
 - Works for classification, but fails for pixel-wise prediction.
- **Goal:**
Keep convolutional feature extraction,
remove the FC bottleneck,
and recover spatial correspondence.

The Concept of Dense Prediction

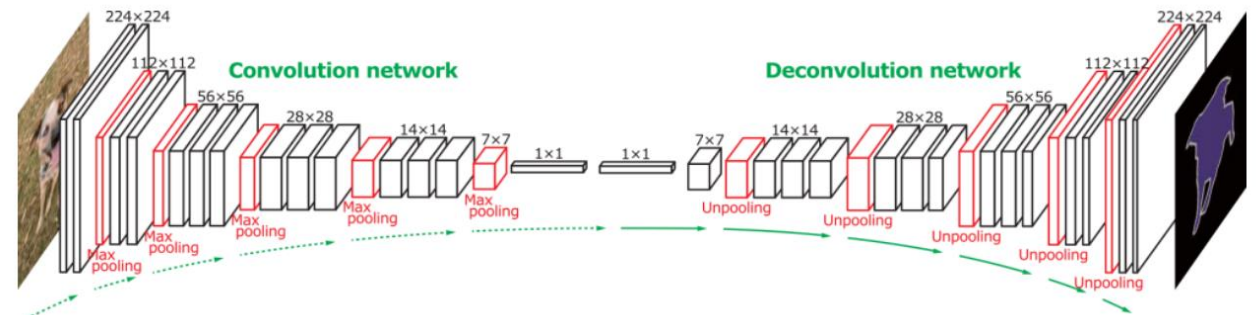
Dense vs. Global Prediction

Task Type	Example	Output	Network Goal
Global Prediction	Image classification	1 label	Is there a cat?
Dense Prediction	Semantic segmentation	Label per pixel	Where is the cat?

Instead of producing one number (or class),
the model must output a grid of values: a 2-D map aligned with the input image.

Fully Convolutional Networks (FCNs)

- Replace fully connected layers with **convolutions**.
- Output is not a single class but a **pixel-level map**.
- **Applications:**
 - Semantic segmentation
 - Depth estimation
 - Object boundary detection



<https://tjmachinelearning.com/lectures/1718/fcn/>

